

| STAT214 Chapter 10 Extending the Least Squares Algorithm

| 1 Linear Fits with Multiple Predictive Features

| 1.1 Interpreting the Coefficients of a Linear Fit

When two variables convey similar information to one another, they are considered to be “collinear.” Since collinearity is a common cause of instability in linear fits, it is important to be wary of placing too much stock in your interpretations of coefficient values when there exist predictor variables that convey related information

| 1.2 Comparing Coefficients

We should refrain from comparing the coefficient values to one another unless the variables

- are independent (that is, they are not collinear)
- are either on the same scale as one another or the coefficient values themselves have been standardized

Dividing each coefficient value by its SD leads to standardized coefficient values, called “**t -values**,” that are comparable with one another:

$$t_j = \frac{b_j}{SD(b_j)}$$

| 1.2.1 Different Ways to Estimate the SD of the Coefficients

- The traditional *theoretical* approach of estimating a coefficient's SD involves estimating how much the coefficient's value varies across alternative *theoretical* versions of the data from a hypothetical data-generating mechanism
- However, we prefer to explicitly create plausible alternative versions of our data using perturbations, e.g., using the **bootstrap** perturbation technique.

| 1.2.2 Using the Bootstrap to Estimate the SD

Using the bootstrap to estimate the SD of a LS coefficient involves the following steps:

1. Create N (e.g., $N=100$ or $N=1,000$) bootstrapped versions of the original dataset so that each of the N bootstrapped datasets has the same number of observations as the original data.
2. For each of the N bootstrapped datasets, compute an LS fit and extract the relevant coefficient value (so that you have N versions of each coefficient value).
3. Compute the SD of the N bootstrapped coefficients.

To empirically standardize the coefficient values using the bootstrap, we can thus divide the original coefficients (based on the original dataset), b_j , by its bootstrapped SD:

$$t_j^{\text{boot}} = \frac{b_j}{SD^{\text{boot}}(b_j)}$$

| 1.2.3 Larger Standardized Coefficient Value

a larger standardized coefficient value

- not only tells us that the corresponding predictive feature is an important predictor of the response
- but also that the coefficient is fairly stable

| 2 Pre-processing: One-Hot-Encoding

| 2.1 One-Hot-Encoding

For any given observation,

- the value of each binary one-hot encoded variable that *does not* correspond to the observation's categorical value (level) will equal 0,
- whereas the binary one-hot encoded variable that *does* correspond to the observation's categorical value will equal 1.

One categorical level is typically chosen to be the “**reference**” value. This reference binary one-hot encoded variable is *removed* from the linear fit so that each of the one-hot encoded variable coefficients can be interpreted as relative to the reference

| 3 Pre-processing: Variable Transformations

| 3.1 LS and Gaussianity

Transformations that improve the symmetry of our variables, such as **log-transformations** or **square-root transformations**, often lead to improved LS predictive performance too

⚠ Remark: log-transformation ▾

Note that log-transformations (and square-root transformations) can be used only for variables with *positive* values. If the variable that you are trying to transform contains negative values, you will need to pre-transform it so that all its values are positive (e.g., by adding a constant value corresponding to a quantity that is slightly larger than the smallest value in the variable).

| 3.2 Heteroskedasticity

The LS algorithm typically performs best when the variability associated with each observation is relatively consistent:

- certain transformations, such as a **log-transformation** of the response, can help reduce heteroskedasticity
- modify the LS algorithm so it takes the heteroskedasticity into account, such as using a technique called *weighted* Least Squares

| 3.3 Fitting Nonlinear Curves

Transformations can also be helpful when you don't believe that the relationship between your response and a predictor variable is linear.

| 4 Feature Selection

| 4.1 Complexity of Predictive Fit

- as the complexity of your predictive fit starts to become too great relative to the amount of training data you have, the algorithm will begin to **overfit** to the training data
- you should have at **least 10 or 20** (or 50, depending on whom you ask) training observations for every predictive feature to avoid overfitting
- the simple solution when you have too many predictive features relative to the number of observations is to conduct **feature selection**

| 4.2 Correlation Screening

Keep only the predictive features that are **most correlated** with the response (based on some cutoff)

⚠ Remark ▾

This technique will typically be most effective for problems where both the response and predictor variables are continuous numeric variables.

Other metrics traditionally used in statistics include the Akaike Information Criterion (AIC), and the Bayesian Information Criterion (BIC)

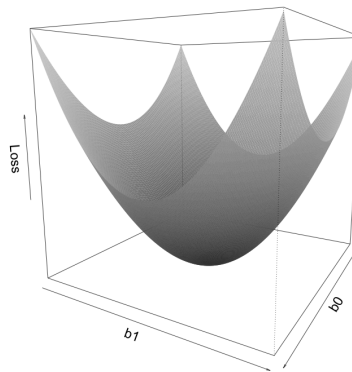
| 5 Regularization

| 5.1 Regularization

- Regularizing a predictive algorithm involves forcing it toward a simpler solution by adding constraints to the minimization/optimization problem, which tends to have the added effect of improving the stability of the algorithm.
- Regularization can be used to address both problems of [overfitting](#) and [collinearity](#)

| 5.2 Ridge

The technique that imposes a quadratic (squared) L2 constraint or penalty term is called **L2 regularization**, and, when used with the LS algorithm, it is called the **ridge** algorithm.



Form of constrained LS problem:

Find the values of b_0 and b_1 that make the LS loss

$$\sum_i (\text{observed price}_i - (b_0 + b_1 \text{area}_i))^2 \quad (10.5)$$

as small as possible, subject to $b_0^2 + b_1^2 < t$ for some $t \geq 0$.

This final expression, $b_0^2 + b_1^2 < t$, is the **L2 constraint** (L2 because the coefficients are squared), and t is called the **regularization hyperparameter**.⁷

The smaller the value of the hyperparameter t , the [more](#) regularization you are implementing

Another form:

Find the values of b_0 and b_1 that make the regularized LS loss

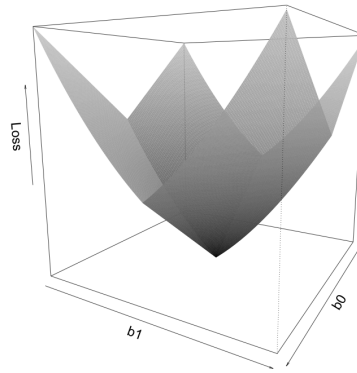
$$\sum_i (\text{observed price}_i - (b_0 + b_1 \text{area}_i))^2 + \lambda(b_0^2 + b_1^2) \quad (10.6)$$

as small as possible (for some $\lambda \geq 0$).

- The λ hyperparameter for this penalized version of the ridge algorithm and the t hyperparameter for the constrained version of the ridge algorithm are essentially equivalent (they have a 1-1 correspondence)
- But they have different interpretations.
 - For the original constrained formulation, the [smaller](#) the value of t , the [more](#) regularization is being implemented
 - For this penalty-term formulation, the [larger](#) the value of λ , the [more](#) regularization is being implemented.

| 5.3 Lasso

The technique that imposes an absolute value L1 constraint or penalty term is called **L1 regularization**, and, when applied to the LS algorithm, it is called the **lasso** algorithm.



Form of constrained LS problem:

Find the values of b_0 and b_1 that make the LS loss

$$\sum_i (\text{observed price}_i - (b_0 + b_1 \text{area}_i))^2 \quad (10.7)$$

as small as possible, subject to $|b_0| + |b_1| < t$ for some $t \geq 0$.

Another form:

Find the values of b_0 and b_1 that make the regularized LS loss

$$\sum_i (\text{observed price}_i - (b_0 + b_1 \text{area}_i))^2 + \lambda(|b_0| + |b_1|) \quad (10.8)$$

as small as possible (for some $\lambda \geq 0$).

⚠ Remark: Lasso and Ridge

- While L2-regularization (ridge) shrinks the coefficients **toward** zero (without actually yielding coefficients **equal to 0**), L1-regularization (lasso) will often shrink several of the coefficients **all the way** to 0
- It is possible to implement both lasso (L1) and ridge (L2) regularization simultaneously, by implementing **two** constraints or adding **two** penalty terms (one for the squared sum and one for the absolute value sum of the coefficients). This algorithm is called **elastic net**.

5.4 Standardization for Lasso and Ridge

- Since regularization in this context involves adding up the coefficients (squared for ridge and absolute value for lasso) and restricting the size of the coefficients based on this sum, if the original variables are on very different scales, the sum will be dominated by features with larger values.
- regardless of whether you are using ridge, lasso, or elastic net regularization, if your predictor variables are not already on the same scale, it is important to **standardize** them to a common scale before applying regularization

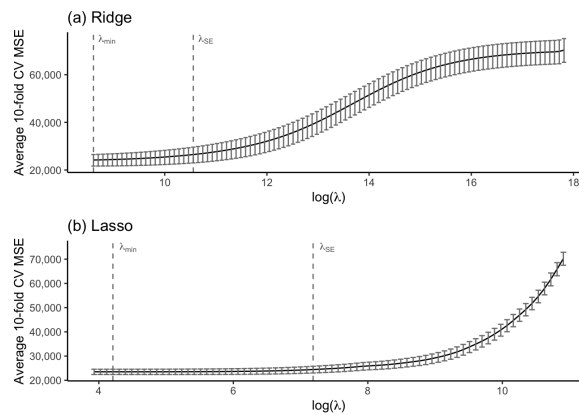
5.5 Cross-Validation for Choosing Regularization Hyperparameters

5.5.1 Procedure

The procedure for using CV to choose a reasonable value of the regularization hyperparameter:

1. Decide on a range of potential values for the penalty term, λ . Note that many software implementations will do this for you.
2. Split the data into V (e.g., $V=10$) nonoverlapping folds of approximately the same size.
3. Remove the first fold (this fold will play the role of the pseudo-validation set), and use the remaining $V-1$ folds (which will play the role of the pseudo-training set) to train the regularized LS fit using each value of λ .

- Calculate the error (e.g., mean squared error (MSE)) for each of the regularized LS fits—one for each λ —using the first withheld CV-fold pseudo-validation set.
- Replace the withheld first fold and now remove the second fold (the second fold will now play the role of the pseudo-validation set). Use the remaining $V - 1$ folds to train the algorithm for each value of λ . Evaluate the fits using the withheld second fold (e.g., using MSE).
- Repeat this process until all the V folds have been used as the withheld validation set, resulting in V measurements of the algorithm's performance for each λ .
- For each value of λ , calculate the average of the V errors. The average of the V errors is called the **CV error**.
- Select the λ that had the lowest CV error, or that you judge to be the best (e.g., taking stability into consideration).



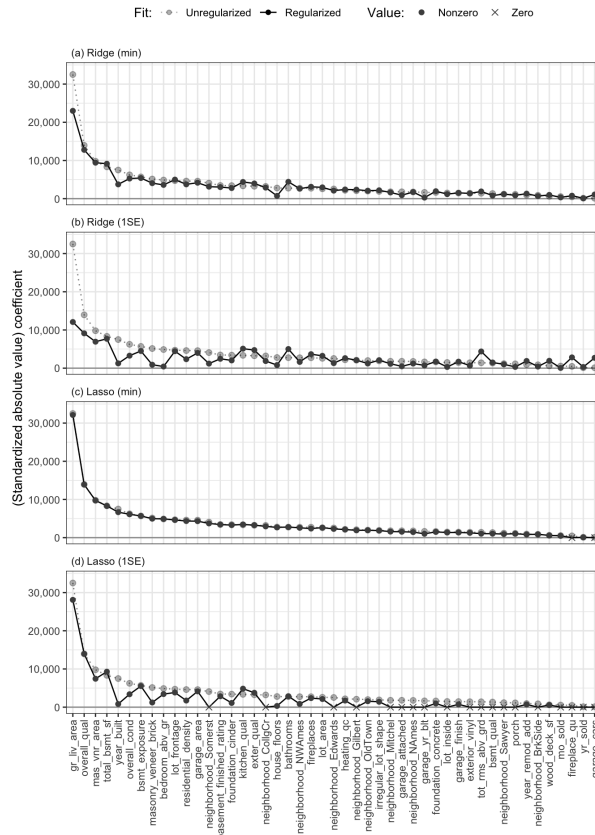
5.5.2 Choose λ

The value that attains the lowest CV error is called $\lambda_{\min}$. However, rather than using the value that attains the smallest CV error,

- it is common to instead opt for a **slightly larger value of λ** , just to enforce a little extra regularization
- another common choice for λ is the largest value for which the MSE is within 1 **standard error (SE)** of the MSE for $\lambda_{\min}$

5.6 Lasso as a Feature Selection Technique

Since the lasso algorithm will often shrink several coefficients all the way to zero, it can be used as a feature selection technique in which you remove all the features with lasso coefficients equal to zero and retrain a linear fit (e.g., using LS) with the remaining features



6 PCS Evaluations

6.1 Predictability

The [validation set](#) performance in terms of root-mean squared error (rMSE), mean absolute error (MAE), and the correlation between the observed and predicted response of possible linear fits

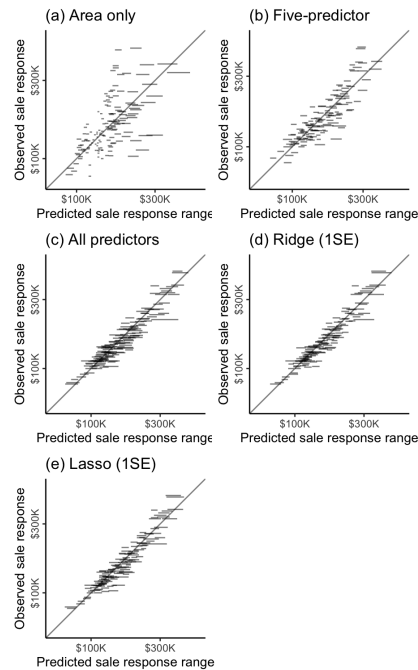
Table 10.2: The rMSE, MAE, and correlation for the validation set predictions generated for the five linear predictive fits. Each fit is trained on one particular cleaned and preprocessed version of the Ames housing training data and is evaluated using the corresponding cleaned and preprocessed version of the validation data

Fit	rMSE	MAE	Correlation
LS (area only)	49,377	34,714	0.696
LS (five-predictor)	31,773	23,858	0.887
LS (all predictors)	23,267	17,332	0.942
Ridge (1SE)	23,607	16,901	0.94
Lasso (1SE)	23,219	16,725	0.941

6.2 Stability

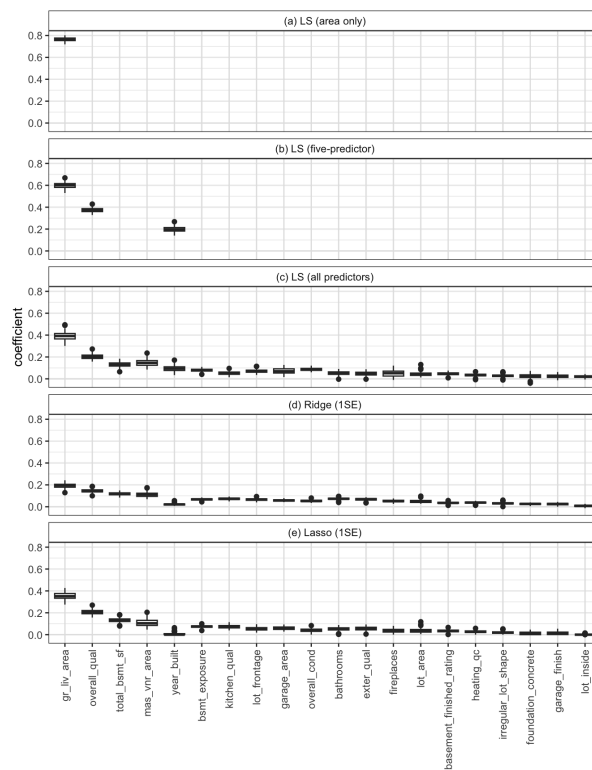
6.2.1 Stability to Data Perturbations

First, we will investigate how stable our predictions are to data perturbations that are designed to reflect how our data may have been collected differently.



6.2.2 Stability of the Coefficients

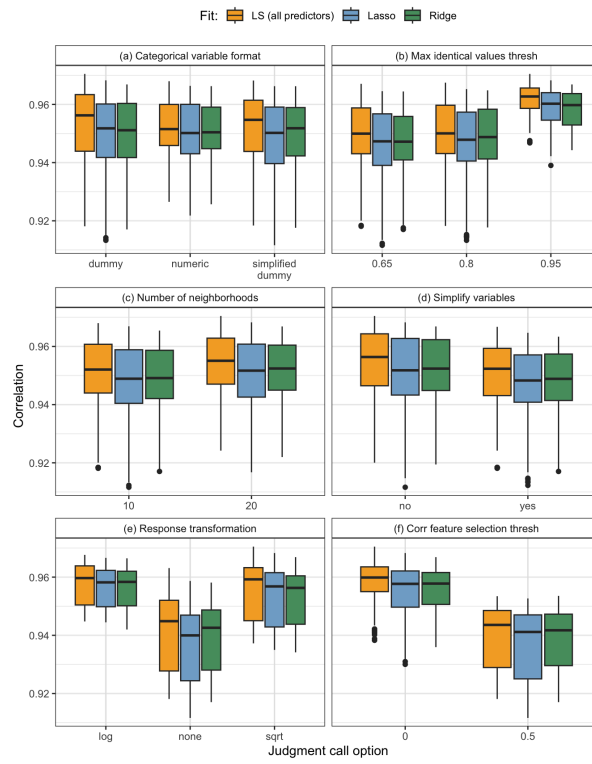
We will next investigate the stability of the [coefficients](#). However, recall that to compare the coefficients to one another, we either need to standardize the coefficients



6.2.3 Stability to Data Cleaning and Pre-processing Judgment Call Perturbations

Investigate how much our predictions change when we make reasonable alternative choices for judgment calls

- Before we begin, we will first conduct **predictability screening** to identify any judgment calls that lead to particularly poor validation set predictive performance



- prediction stability plot

